

Roteiro de Implementação — Formas e Condições de Pagamento

Sistema Gestor • BackEnd Go + BFF Express + FrontEnd React • Documento de estudo com implementação manual passo a passo

0. Entenda a arquitetura

Camada	Tecnologia	Responsabilidade	Pasta
API	Go (chi + pgx)	Regras de acesso ao PostgreSQL, rotas <code>/formaPagamento</code> , <code>/condicaoPagamento</code>	<code>BackEnd\Server\Go\src</code>
BFF	Express (TypeScript)	Proxy autenticado que o browser acessa; valida payload com Zod	<code>FrontEnd\src\server</code>
Cliente	React + Vite	Telas (páginas + formulários), menu e permissões	<code>FrontEnd\src\client</code>

Fluxo: Browser → `/api/formas-pagamento` (Express :3001) → `/formaPagamento` (Go :9000) → PostgreSQL.

Sobre as sequências de ID: não é preciso criar nada no banco para gerar IDs. A função `database.GerarID` faz upsert automático em `empresa_sequences` (por empresa + tabela), inicializando com `MAX(id)+1`. O servidor também roda `SeedSequences` na inicialização e registra tabelas novas sozinho.

1. BackEnd Go — handler de pagamento

1.1 Criar o arquivo `handlers\handler_pagamento.go`

Crie o arquivo no pacote `handlers`. Ele segue o mesmo padrão de `handler_customizacao.go` (Adicional): listagem manual com filtros, gravação via `genericUpsert` e exclusão dedicada. As FKs das suas tabelas têm `ON DELETE CASCADE`, então excluir uma forma/condição limpa as associações automaticamente.

```
package handlers

import (
    "fmt"
    "net/http"

    "github.com/go-chi/chi/v5/middleware"
)

// --- Condição de Pagamento ---
func (h *ProducaoHandler) CondiçãoPagamentoListar(w http.ResponseWriter, r *http.Request) {
    empresaID := middleware.GetEmpresaID(r)
    id := parseInt(r.URL.Query().Get("id"), 0)
    desc := r.URL.Query().Get("descricao")

    query := `SELECT * FROM condicao_pagamento WHERE 1=1`
    var args []interface{}
    argN := 1
    if id > 0 {
        query += fmt.Sprintf(" AND id = %d", argN)
        argN++
        args = append(args, id)
    }
    if desc != "" {
        query += fmt.Sprintf(" AND upper(descricao) LIKE upper(%s)", argN)
        argN++
        args = append(args, "%"+desc+"%")
    }
    query += fmt.Sprintf(" AND empresa_id = %d ORDER BY descricao", argN)
```

```

        args = append(args, empresaID)

        rows, err := h.Pool.Query(r.Context(), query, args...)
        if err != nil {
            jsonError(w, err.Error(), http.StatusInternalServerError)
            return
        }
        defer rows.Close()
        jsonSuccess(w, rowsToMap(rows))
    }

    func (h *ProducaoHandler) CondicaoPagamentoSalvar(w http.ResponseWriter, r *http.Request) {
        h.BasicCRUD.Salvar(w, r, "condicao_pagamento",
            []string{"descricao", "qtd_parcelas", "dias_primeiro_vencimento",
                "dias_intervalo", "status"})
    }

    func (h *ProducaoHandler) CondicaoPagamentoExcluir(w http.ResponseWriter, r *http.Request) {
        h.genericDelete(w, r, "condicao_pagamento")
    }

```

Continua na próxima página...

1.2 Forma de Pagamento + associação com condições (no mesmo arquivo)

```
// --- Forma de Pagamento ---
func (h *ProducaoHandler) FormaPagamentoListar(w http.ResponseWriter, r *http.Request) {
    empresaID := middleware.GetEmpresaID(r)
    id := parseInt(r.URL.Query().Get("id"), 0)

    query := `SELECT * FROM forma_pagamento WHERE 1=1`
    var args []interface{}
    argN := 1
    if id > 0 {
        query += fmt.Sprintf(" AND id = %d", argN)
        argN++
        args = append(args, id)
    }
    query += fmt.Sprintf(" AND empresa_id = %d ORDER BY descricao", argN)
    args = append(args, empresaID)

    rows, err := h.Pool.Query(r.Context(), query, args...)
    if err != nil {
        jsonError(w, err.Error(), http.StatusInternalServerError)
        return
    }
    defer rows.Close()
    jsonSuccess(w, rowsToMap(rows))
}

func (h *ProducaoHandler) FormaPagamentoSalvar(w http.ResponseWriter, r *http.Request) {
    h.BasicCRUD.Salvar(w, r, "forma_pagamento",
        []string{"descricao", "classificacao", "status"})
}

func (h *ProducaoHandler) FormaPagamentoExcluir(w http.ResponseWriter, r *http.Request) {
    h.genericDelete(w, r, "forma_pagamento")
}

// FormaPagamentoCondicaoListar lista as condicoes vinculadas a uma forma.
// GET /formaPagamentoCondicao?forma_pagamento_id=10
func (h *ProducaoHandler) FormaPagamentoCondicaoListar(w http.ResponseWriter, r *http.Request) {
    empresaID := middleware.GetEmpresaID(r)
    formaID := parseInt(r.URL.Query().Get("forma_pagamento_id"), 0)
    if formaID == 0 {
        jsonError(w, "forma_pagamento_id não informado", http.StatusBadRequest)
        return
    }
    rows, err := h.Pool.Query(r.Context(), `
        SELECT fpc.forma_pagamento_id, fpc.condicao_pagamento_id, cp.descricao,
               cp.qtd_parcelas, cp.dias_primeiro_vencimento, cp.dias_intervalo
        FROM forma_pagamento_condicao fpc
        JOIN condicao_pagamento cp
          ON cp.id = fpc.condicao_pagamento_id AND cp.empresa_id = fpc.empresa_id
        WHERE fpc.empresa_id = $1 AND fpc.forma_pagamento_id = $2 AND fpc.status = 1
        ORDER BY cp.descricao`, empresaID, formaID)
    if err != nil {
        jsonError(w, err.Error(), http.StatusInternalServerError)
        return
    }
    defer rows.Close()
    jsonSuccess(w, rowsToMap(rows))
}

// FormaPagamentoCondicaoSalvar substitui o vinculo da forma pelo array enviado.
// POST /formaPagamentoCondicao body: {"forma_pagamento_id":10,"condicoes":[1,2,3]}
func (h *ProducaoHandler) FormaPagamentoCondicaoSalvar(w http.ResponseWriter, r *http.Request) {
```

```

items, err := h.BasicCRUD.parseBody(r)
if err != nil {
    jsonError(w, err.Error(), http.StatusBadRequest)
    return
}
empresaID := middleware.GetEmpresaID(r)

header := items[0]
formaID := getInt(header, "forma_pagamento_id")
if formaID == 0 {
    jsonError(w, "forma_pagamento_id é obrigatório", http.StatusBadRequest)
    return
}
rawConds, _ := header["condicoes"].([]interface{})

tx, err := h.Pool.Begin(r.Context())
if err != nil {
    jsonError(w, "Erro interno", http.StatusInternalServerError)
    return
}
defer tx.Rollback(r.Context())

if _, err = tx.Exec(r.Context(),
    `DELETE FROM forma_pagamento_condicao WHERE empresa_id = $1 AND forma_pagamento_id = $2`,
    empresaID, formaID); err != nil {
    jsonError(w, err.Error(), http.StatusInternalServerError)
    return
}

for _, raw := range rawConds {
    condID := int(raw.(float64))
    if _, err = tx.Exec(r.Context(), `
        INSERT INTO forma_pagamento_condicao
            (empresa_id, forma_pagamento_id, condicao_pagamento_id, status)
        VALUES ($1,$2,$3,1)`, empresaID, formaID, condID); err != nil {
        jsonError(w, err.Error(), http.StatusInternalServerError)
        return
    }
}

if err = tx.Commit(r.Context()); err != nil {
    jsonError(w, err.Error(), http.StatusInternalServerError)
    return
}

jsonSuccess(w, map[string]interface{}{"mensagem": "Condições da forma salvas com sucesso"})
}

```

Onde ficam os utilitários usados acima: `parseInt`, `getInt`, `jsonSuccess`, `jsonError`, `rowsToMap` e `BasicCRUD.parseBody` já existem no projeto (`handlers\handlers_crud.go` e `handlers\crud_basico.go`). `middleware.GetEmpresaID(r)` lê a empresa do token JWT. `BasicCRUD.Salvar` gera ID via `GerarID` quando o JSON vem sem `id` (INSERT) ou faz UPDATE quando vem com — por isso se chama "upsert".

1.3 Registrar as rotas no `main.go`

Junto do bloco de rotas de produção (por volta da linha 429, dentro do grupo autenticado — o mesmo onde estão `/adicional` e `/categoriaPagar`), adicione:

```
// Pagamento (formas e condicoes)
r.Get("/formaPagamento", producao.FormaPagamentoListar)
r.Post("/formaPagamento", producao.FormaPagamentoSalvar)
r.Delete("/formaPagamento", producao.FormaPagamentoExcluir)

r.Get("/condicaoPagamento", producao.CondicaoPagamentoListar)
r.Post("/condicaoPagamento", producao.CondicaoPagamentoSalvar)
r.Delete("/condicaoPagamento", producao.CondicaoPagamentoExcluir)

r.Get("/formaPagamentoCondicao", producao.FormaPagamentoCondicaoListar)
r.Post("/formaPagamentoCondicao", producao.FormaPagamentoCondicaoSalvar)
```

A variável `producao` é a instância de `ProducaoHandler` já declarada no `main.go` (mesma usada pelas rotas de adicional). Todas as rotas exigem o header `Authorization: Bearer <token>`.

1.4 Compilar e reiniciar a API Go

```
cd C:\Users\mjtam\developer\projects\Gestor\BackEnd\Server\Go\src
go build -o main-novo.exe ./cmd/...
# reinicie o servidor na porta 9000 com o binário novo
```

Teste rápido antes de seguir: gere um token (login) e chame `curl -H "Authorization: Bearer TOKEN" http://localhost:9000/condicaoPagamento`. Deve responder JSON vazio (`[]`) sem erro 500.

2. BFF Express (FrontEnd\src\server)

2.1 Tipos em src\types.ts

```
export interface CondicaoPagamento {
  empresa_id?: number;
  id?: number;
  descricao: string;
  qtd_parcelas: number;
  dias_primeiro_vencimento: number;
  dias_intervalo: number;
  status?: number;
}

export interface FormaPagamento {
  empresa_id?: number;
  id?: number;
  descricao: string;
  classificacao?: 'DINHEIRO' | 'PIX' | 'CARTAO_CREDITO' | 'CARTAO_DEBITO'
    | 'BOLETO' | 'CREDIARIO' | 'BANCO' | 'CHEQUE_A_VISTA'
    | 'CHEQUE_A_PRAZO' | 'OUTROS';
  status?: number;
}

export interface FormaPagamentoCondicao {
  forma_pagamento_id: number;
  condicao_pagamento_id: number;
  descricao?: string;
  qtd_parcelas?: number;
}
```

2.2 Schemas Zod em src\schemas\index.ts

Siga o padrão de `adicionalBodySchema`: aceita um objeto OU array de objetos (a API Go faz upsert item a item).

```
const condicaoPagamentoItemSchema = z.object({
  id: z.number().int().positive().optional(),
  descricao: z.string().min(1, 'Descricao obrigatoria').max(200),
  qtd_parcelas: z.number().int().positive('Qtd de parcelas obrigatoria'),
  dias_primeiro_vencimento: z.number().int().min(0),
  dias_intervalo: z.number().int().min(0),
  status: z.number().int().optional(),
});
export const condicaoPagamentoBodySchema = z.union([
  condicaoPagamentoItemSchema,
  z.array(condicaoPagamentoItemSchema).min(1),
]);

const formaPagamentoItemSchema = z.object({
  id: z.number().int().positive().optional(),
  descricao: z.string().min(1, 'Descricao obrigatoria').max(200),
  classificacao: z.string().max(30).optional(),
  status: z.number().int().optional(),
});
export const formaPagamentoBodySchema = z.union([
  formaPagamentoItemSchema,
  z.array(formaPagamentoItemSchema).min(1),
]);

export const formaPagamentoCondicaoBodySchema = z.object({
  forma_pagamento_id: z.number().int().positive('Forma e obrigatoria'),
  condicoes: z.array(z.number().int().positive()).default([]),
});
```

2.3 Métodos no `src\services\horseApi.ts`

Espelhe `listarAdicionais/salvarAdicionais/excluirAdicional`:

```
// Condicoes
async listarCondicoesPagamento(params?: { id?: number; descricao?: string }) {
  const res = await this.api.get('/condicaoPagamento',
    { params, headers: this.getAuthHeaders() });
  return res.data;
}

async salvarCondicoesPagamento(items: unknown) {
  const res = await this.api.post('/condicaoPagamento', items,
    { headers: this.getAuthHeaders() });
  return res.data;
}

async excluirCondicaoPagamento(id: number) {
  const res = await this.api.delete('/condicaoPagamento',
    { params: { id }, headers: this.getAuthHeaders() });
  return res.data;
}

// Formas
async listarFormasPagamento(params?: { id?: number; descricao?: string }) {
  const res = await this.api.get('/formaPagamento',
    { params, headers: this.getAuthHeaders() });
  return res.data;
}

async salvarFormasPagamento(items: unknown) {
  const res = await this.api.post('/formaPagamento', items,
    { headers: this.getAuthHeaders() });
  return res.data;
}

async excluirFormaPagamento(id: number) {
  const res = await this.api.delete('/formaPagamento',
    { params: { id }, headers: this.getAuthHeaders() });
  return res.data;
}

// Associacao forma -> condicoes
async listarFormaPagamentoCondicoes(formaPagamentoId: number) {
  const res = await this.api.get('/formaPagamentoCondicao',
    { params: { forma_pagamento_id: formaPagamentoId }, headers: this.getAuthHeaders() });
  return res.data;
}

async salvarFormaPagamentoCondicoes(body: unknown) {
  const res = await this.api.post('/formaPagamentoCondicao', body,
    { headers: this.getAuthHeaders() });
  return res.data;
}
```

2.4 Criar `src\routes\condicoes-pagamento.ts`

Cópia do estilo de `adicionais.ts` (DELETE por query `?id=`, erro com status propagado):

```
import { Router, Response } from 'express';
import { horseApi } from '../services/horseApi';
import { authMiddleware, AuthRequest } from '../middleware/auth';
import { validate } from '../middleware/validate';
import { condicaoPagamentoBodySchema } from '../schemas';

const router = Router();

router.get('/', authMiddleware, async (req: AuthRequest, res: Response) => {
  try {
    const result = await horseApi.listarCondicoesPagamento(
      req.query as Record<string, unknown>);
```

```

    res.json(result);
  } catch (error: unknown) {
    const status = error instanceof Error && 'status' in error
      ? (error as { status: number }).status : 500;
    res.status(status).json({ error: error instanceof Error ? error.message : 'Erro interno' });
  }
});

router.post('/', authMiddleware, validate(condicaoPagamentoBodySchema),
  async (req: AuthRequest, res: Response) => {
    try {
      const items = Array.isArray(req.body) ? req.body : [req.body];
      const result = await horseApi.salvarCondicoesPagamento(items);
      res.json(result);
    } catch (error: unknown) {
      const status = error instanceof Error && 'status' in error
        ? (error as { status: number }).status : 500;
      res.status(status).json({ error: error instanceof Error ? error.message : 'Erro interno' });
    }
  });

router.delete('/', authMiddleware, async (req: AuthRequest, res: Response) => {
  try {
    const { id } = req.query;
    if (!id) { res.status(400).json({ error: 'ID e obrigatorio' }); return; }
    const result = await horseApi.excluirCondiacaoPagamento(Number(id));
    res.json(result);
  } catch (error: unknown) {
    const status = error instanceof Error && 'status' in error
      ? (error as { status: number }).status : 500;
    res.status(status).json({ error: error instanceof Error ? error.message : 'Erro interno' });
  }
});

export default router;

```

Dica: os arquivos `formas-pagamento.ts` e `formas-pagamento-condicoes.ts` seguem exatamente a mesma estrutura, trocando schema e métodos do `horseApi` (`listarFormasPagamento/salvarFormasPagamento/excluirFormaPagamento` e `listarFormaPagamentoCondicoes/salvarFormaPagamentoCondicoes`). No router de associação, o GET lê `req.query.forma_pagamento_id` (400 se ausente) e o POST valida `formaPagamentoCondiacaoBodySchema` .

2.5 Montar os routers em `src\index.ts`

```

// junto dos outros imports de rotas (~linha 30)
import condicoesPagamentoRoutes from './routes/condicoes-pagamento';
import formasPagamentoRoutes from './routes/formas-pagamento';
import formasPagamentoCondicoesRoutes from './routes/formas-pagamento-condicoes';

// junto dos outros app.use('/api/...', ...) (~linha 141)
app.use('/api/condicoes-pagamento', condicoesPagamentoRoutes);
app.use('/api/formas-pagamento', formasPagamentoRoutes);
app.use('/api/formas-pagamento-condicoes', formasPagamentoCondicoesRoutes);

```

Reinicie o BFF (`npm run dev` dentro de `FrontEnd\src\server`).

3. Cliente React (FrontEnd\src\client)

3.1 Tipos em src\types\index.ts

Repita as mesmas interfaces `CondicaoPagamento`, `FormaPagamento` e `FormaPagamentoCondicao` da seção 2.1 (o cliente tem o próprio arquivo de tipos, sem import do server).

3.2 Tela de Condições: src\pages\CondicoesPagamento.tsx

Use `Adicionais.tsx` como gabarito (mesmos imports: `PaginaFiltros`, `DataTable`, `Modal`, `ConfirmDialog`, `PageHeader`, `RowActions`, `ShowForPermission`). Estrutura essencial:

```
export function CondicoesPagamento() {
  const { data, loading, error, create, update, remove, refetch }
    = useApi<CondicaoPagamento>('/condicoes-pagamento');
  const [busca, setBusca] = useState('');
  const [filtroAtivo, setFiltroAtivo] = useState<FiltroStatusAtivo>('1');
  const [modalOpen, setModalOpen] = useState(false);
  const [editing, setEditing] = useState<CondicaoPagamento | null>(null);
  const [confirmDelete, setConfirmDelete] = useState<number | null>(null);
  const { addToast } = useToast();

  const filtradas = useMemo(() => (data ?? []).filter((c) =>
    passaStatusAtivo(c.status === 0 ? false : true, filtroAtivo)
    && passaBusca([c.descricao], busca)), [data, filtroAtivo, busca]);

  // colunas com createColumnHelper<CondicaoPagamento>():
  //   id | descricao | qtd_parcelas ("1x") | intervalo ("30/60/90") | status badge

  async function handleSave(form: CondicaoPagamento) {
    try {
      if (form.id) await update(form.id, form);
      else await create(form);
      addToast({ message: 'Condição salva!', type: 'success' });
      setModalOpen(false);
      setEditing(null);
    } catch {
      addToast({ message: 'Erro ao salvar', type: 'error' });
    }
  }

  return (
    <Layout>
      <PageHeader title="Condições de Pagamento"
        actions={<>
          <ShowForPermission rota="/condicoes-pagamento" acao={ACAO.INCLUIR}>
            <Button onClick={() => { setEditing(null); setModalOpen(true); }}>
              <Plus size={16} /> Novo
            </Button>
          </ShowForPermission>
        </> />
      <Card>
        <PaginaFiltros busca={busca} setBusca={setBusca}
          filtroAtivo={filtroAtivo} setFiltroAtivo={setFiltroAtivo} />
        { /* DataTable com RowActions (editar/excluir) por linha */ }
      </Card>
      <Modal isOpen={modalOpen} onClose={() => setModalOpen(false)}
        title={editing ? 'Editar Condição' : 'Nova Condição'}>
        <CondicaoPagamentoForm initialData={editing} onSubmit={handleSave}
          onCancel={() => setModalOpen(false)} />
      </Modal>
      <ConfirmDialog /* excluir via remove(id) */ />
    </Layout>
  )
}
```

```
    );  
  }
```

3.3 Formulário: `src\components\forms\CondicaoPagamentoForm.tsx`

Espelhe `AdicionalForm.tsx`: estado local inicializado de `initialData`, campos controlados, validação simples (descrição obrigatória), botões Salvar/Cancelar chamando `onSubmit` / `onCancel`.

```
<input placeholder="Descrição" />  
<label>Qtd de parcelas: <input type="number" min={1} /></label>  
<label>Dias do 1º vencimento: <input type="number" min={0} /></label>  
<label>Intervalo entre parcelas (dias): <input type="number" min={0} /></label>  
<label><input type="checkbox" /> Ativo</label>
```

Continua na próxima página...

3.4 Tela de Formas com associação: `src\pages\FormasPagamento.tsx`

É a parte mais interessante: copie o padrão de **linhas expansíveis** do `Adicionais.tsx` (estado `links`, `fetchLinks`, `renderSubComponent`). As diferenças:

```
// carregar condicoes vinculadas de uma forma
const res = await api.get<FormaPagamentoCondicao[]>('/formas-pagamento-condicoes',
  { params: { forma_pagamento_id: pid } });

// modal de vinculo: lista TODAS as condicoes ativas com checkbox
const { data: todas } = useApi<CondicaoPagamento>('/condicoes-pagamento');

// salvar vinculo (substitui todos os vinculos da forma)
await api.post('/formas-pagamento-condicoes', {
  forma_pagamento_id: pid,
  condicoes: selecionadas.map((c) => c.id),
});
refetch(); // recarrega a grade

// coluna extra na grade mostrando resumo dos vinculos:
// "À vista, 30/60" (a partir de links[forma.id])
```

3.5 Rotas em `src\App.tsx`

```
// lazy imports junto dos demais (~linha 41)
const CondicoesPagamento = lazy(() =>
  import('@/pages/CondicoesPagamento').then((m) => ({ default: m.CondicoesPagamento })));
const FormasPagamento = lazy(() =>
  import('@/pages/FormasPagamento').then((m) => ({ default: m.FormasPagamento })));

// rotas junto das demais de gestor (~linha 159)
{routesFor(GESTOR, mode) &&
  <Route path="/condicoes-pagamento" element={<Private><CondicoesPagamento /></Private>} />}
{routesFor(GESTOR, mode) &&
  <Route path="/formas-pagamento" element={<Private><FormasPagamento /></Private>} />}
```

3.6 Mapa de formulários: `src\lib\permissions.ts`

```
// dentro de formRouteMap – a chave deve ser IDÊNTICA ao nome
// que será cadastrado no banco (etapa 4)
'Formas de Pagamento': '/formas-pagamento',
'Condições de Pagamento': '/condicoes-pagamento',
```

3.7 Agrupamento no menu: `src\components\ui\Sidebar.tsx`

Adicione os dois nomes em `moduleSubGroups`, grupo `Geral > Cadastro` (mesma linha de 'Clientes', 'Fornecedores'):

```
forms: ['Clientes', 'Fornecedores', 'Usuarios', 'Marcas',
  'Formas de Pagamento', 'Condições de Pagamento'],
```

4. Cadastro dos formulários e permissões (banco)

O menu e as permissões são dirigidos pelo banco. Nas telas administrativas:

1. **Configurações do Sistema** → **Formulários**: crie os registros "Formas de Pagamento" e "Condições de Pagamento".
2. **Módulo x Formulário**: vincule os dois formulários ao módulo usado no modo Gestor.
3. **Usuário x Formulário**: conceda ao seu usuário as ações INCLUIR / EDITAR / EXCLUIR (é isso que alimenta `ShowForPermission` e os botões).
4. Refaça login para o menu recarregar.

5. Teste ponta a ponta

#	Cenário	Resultado esperado
1	Criar condição "À vista" (1 parcela, 0 dias) pela tela	Linha aparece na grade; SELECT em <code>condicao_pagamento</code> mostra registro com id gerado por sequência
2	Criar forma "PIX" (classificação PIX)	Aparece na grade de formas
3	Vincular "À vista" e "30/60" à forma PIX	Sub-linha expansível lista as duas; tabela <code>forma_pagamento_condicao</code> tem 2 linhas com status 1
4	Romper um vínculo e regravar	POST substitui: DELETE + INSERT dentro da transação; nenhuma linha órfã
5	Inativar/Excluir a forma PIX	Filhas somem via ON DELETE CASCADE (ou status 0 se inativação)
6	Usuário sem permissão abre a tela	Menu não exibido; botões Novo/Editar/Excluir ocultos (<code>ShowForPermission</code>)

6. Checklist de arquivos

Camada	Arquivo	Ação
Banco	forma_pagamento / condicao_pagamento / forma_pagamento_condicao	Já criadas ✓
Go API (:9000)	<code>handlers\handler_pagamento.go</code>	Criar (8 handlers)
	<code>main.go</code>	8 rotas r.Get/r.Post/r.Delete
	build <code>go build ./...</code>	Reiniciar servidor
BFF (:3001)	<code>src\types.ts</code>	3 interfaces
	<code>src\schemas\index.ts</code>	3 schemas Zod
	<code>src\services\horseApi.ts</code>	8 métodos
	<code>src\routes\condicoes-pagamento.ts</code> (+2 arquivos irmãos)	Criar 3 routers
	<code>src\index.ts</code>	3 imports + 3 app.use
React	<code>src\types\index.ts</code>	3 interfaces
	<code>src\pages\CondicoesPagamento.tsx</code> / <code>FormasPagamento.tsx</code>	Criar telas
	<code>src\components\forms\CondicaoPagamentoForm.tsx</code> (+ <code>FormaPagamentoForm</code>)	Criar forms
	<code>src\App.tsx</code> + <code>lib\permissions.ts</code> + <code>Sidebar.tsx</code>	Rotas, formRouteMap, menu
	build <code>npm run build</code> (client)	Validar TypeScript

Ordem sugerida de implementação: Go (1) → teste com curl (1.4) → BFF (2) → teste pelo devtools/network do navegador (2.5) → telas React (3) → permissões (4) → testes (5). Implemente uma camada por vez validando antes de avançar.